Université de Béjaïa

Cycle ingénieur — 4e année Informatique

Spécifications fonctionnelles et techniques

VMPGC

Compilateur Toy et machine virtuelle
à ramasse-miettes précis

Projet 4 : machine virtuelle avec ramasse-miettes (*Garbage Collector*) précis

Rédigé par : Mansour Abderezak
Badreddine Mamouni

Dépôt : https://github.com/bizkda/VMPGC

Version 1.0 — 27 octobre 2026
Année universitaire 2026–2027

Table des matières

1 Présentation du document **3**
1.1 Objet . . . 3
1.2 Rédacteurs et historique des mises à jour . . . 3
1.3 Documents de référence . . . 3
1.4 Conventions . . . 3

2 Définition du périmètre fonctionnel **3**

3 Profils d'utilisateurs et cas d'utilisation **4**

4 Arborescence fonctionnelle **4**

5 Priorités et dépendances **4**

6 Spécification du langage Toy **6**
6.1 Éléments lexicaux . . . 6
6.2 Types et valeurs . . . 6
6.3 Déclarations . . . 6
6.4 Règles de typage . . . 6
6.5 Sémantique d'exécution . . . 7

7 Spécification du compilateur (F1, F4.1) **7**
7.1 Interface en ligne de commande . . . 7
7.2 Chaîne de traitement . . . 8
7.3 Messages d'erreur . . . 8

8 Spécification du bytecode et du fichier .tbc (F1.4, F2.1) **9**
8.1 Modèle d'exécution . . . 9
8.2 Jeu d'instructions (proposé, arrêté à M3) . . . 9
8.3 Format du fichier .tbc . . . 10
8.4 Stack maps . . . 10

9 Spécification de la VM, du tas et du GC (F2, F3, F4.2) **11**
9.1 Interface en ligne de commande . . . 11
9.2 Codes de sortie . . . 11
9.3 Disposition des objets (proposé, arrêté à M4) . . . 11
9.4 Allocation et déclenchement du GC . . . 12
9.5 Racines et précision . . . 12
9.6 Mark-and-Sweep (F2.4) . . . 12
9.7 Stop-and-Copy, algorithme de Cheney (F2.5) . . . 12
9.8 Vérificateur de tas (F3.2) . . . 13

9.9 Statistiques (F3.3) 13

10 Contraintes et règles de gestion **13**

11 Traçabilité avec le cahier des charges **14**

A Illustration : une fonction compilée **14**

1 Présentation du document

1.1 Objet

Ce document décrit précisément ce que fait le système VMPGC : le langage Toy accepté par le compilateur, le comportement du compilateur, le bytecode et son format de fichier, puis le comportement de la machine virtuelle et de ses deux ramasse-miettes. Il répond au cahier des charges, qui définit le besoin et le périmètre, et sert de référence pour écrire le code et les tests.

1.2 Rédacteurs et historique des mises à jour

Version	Date	Rédacteurs	Modification
1.0	27 octobre 2026	Mansour Abderezak, Badreddine Mamouni	Création du document

1.3 Documents de référence

— Cahier des charges VMPGC, version 1.0 (noté *CDC* dans la suite).

— Sujet du projet 4 : machine virtuelle avec ramasse-miettes précis.

1.4 Conventions

— Les identifiants sont stables pour permettre la traçabilité : **F** (fonctionnalité), **UC** (cas d'utilisation), **T** (règle de typage), **RG** (règle de gestion).

— « doit » exprime une exigence obligatoire ; « peut » une option.

— Les valeurs marquées *(proposé)* sont des choix de conception à confirmer pendant le jalon indiqué.

— Les entiers du bytecode et du format de fichier sont en little-endian.

2 Définition du périmètre fonctionnel

Le périmètre est défini avec la démarche de l'*impact mapping*, en répondant à quatre questions.

Question	Réponse
Quel est l'objectif du logiciel ?	Compiler des programmes Toy en bytecode et les exécuter sur une machine virtuelle dont le ramasse-miettes récupère la mémoire inutile de façon précise, sans jamais libérer un objet vivant.
Qui sont les utilisateurs finaux ?	Le développeur Toy, l'évaluateur (encadrant) et les auteurs du projet.
Quels bénéfices recherchent-ils ?	Écrire des programmes sans gérer la mémoire à la main ; vérifier que le GC est correct ; comparer Mark-and-Sweep et Stop-and-Copy avec des mesures.

Question	Réponse
Quelles fonctionnalités y mènent ?	Compilateur Toy (F1), machine virtuelle (F2), outils de vérification et de mesure (F3), interfaces en ligne de commande (F4).

Le périmètre exclu est celui du CDC (code natif, GC générationnel ou concurrent, multithreading, chaînes, flottants, héritage, tableaux multidimensionnels, optimisations).

3 Profils d'utilisateurs et cas d'utilisation

ID	Acteur	Scénario nominal	Scénarios alternatifs
UC1	Développeur Toy	Compile un programme : lance `toyc.py prog.toy`. Le compilateur analyse, vérifie les types, écrit `prog.tbc` et se termine avec le code 0.	Erreur lexicale, syntaxique ou de type : message `ligne:colonne`, aucun `.tbc` produit, code 1. Fichier introuvable : message, code 1.
UC2	Développeur Toy	Exécute un programme compilé : lance `gcvm prog.tbc`. La sortie de `print` s'affiche, la mémoire est récupérée automatiquement.	Fichier `.tbc` invalide : message, code 103. Erreur d'exécution : message sur la sortie d'erreur, code 100.
UC3	Évaluateur	Choisit l'algorithme et la taille du tas avec `-gc` et `-heap`, puis compare les résultats.	Valeur d'option invalide : message d'usage, code 1.
UC4	Évaluateur	Active `-stress` : une collecte a lieu à chaque allocation et le vérificateur contrôle le tas après chaque collecte.	Anomalie détectée : message précis, code 102.
UC5	Évaluateur	Active `-gc-stats` et lit les statistiques de collecte en fin d'exécution.	—
UC6	Auteurs	Lancent la suite de tests : programmes valides (sortie attendue) et invalides (erreur attendue).	Un test échoue : le jalon n'est pas validé.

4 Arborescence fonctionnelle

5 Priorités et dépendances

Les fonctionnalités sont classées avec la méthode MoSCoW : *Must* (indispensable), *Should* (important), *Could* (si le temps le permet).

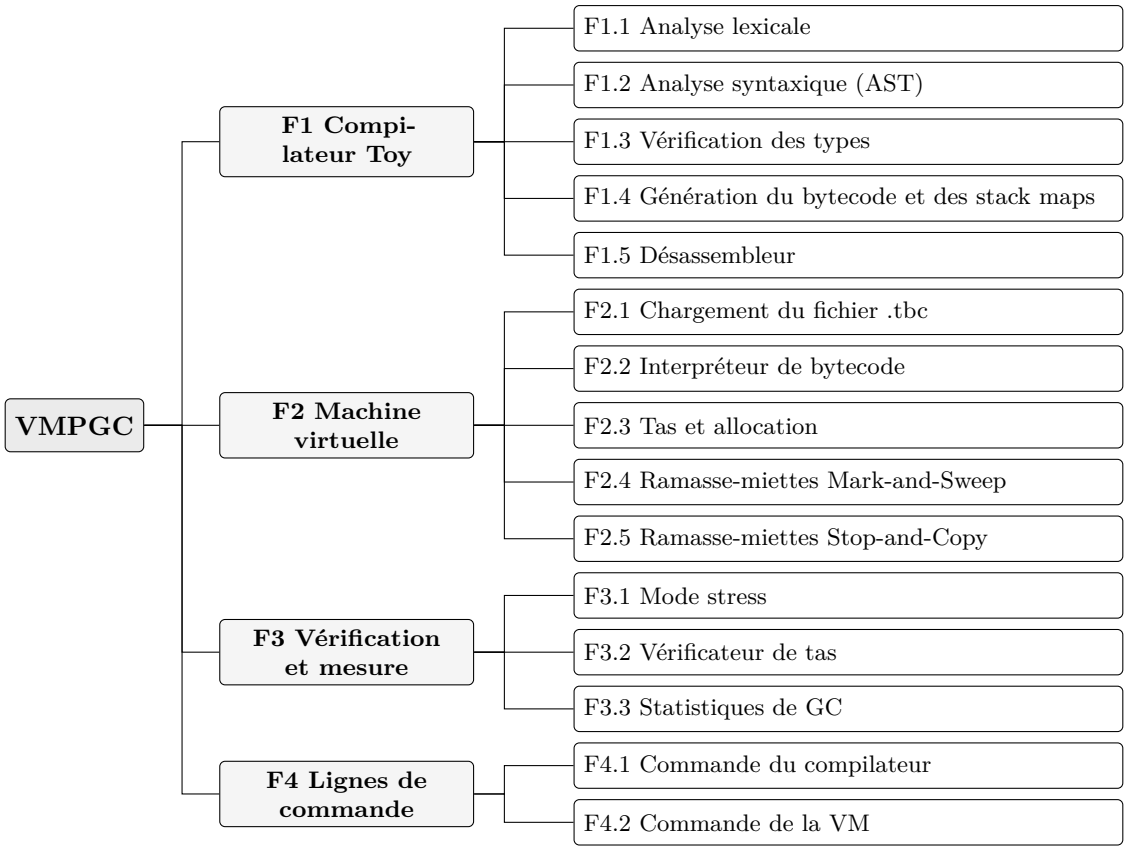


FIGURE 1 – Arborescence des fonctionnalités de VMPGC

ID	Fonctionnalité	Priorité	Dépend de	Jalon
F1.1	Analyse lexicale	Must	—	M1
F1.2	Analyse syntaxique	Must	F1.1	M1
F1.3	Vérification des types	Must	F1.2	M2
F1.4	Génération du bytecode et des stack maps	Must	F1.3	M3
F1.5	Désassembleur	Should	F1.4	M3
F2.1	Chargement du fichier `.tbc`	Must	F1.4	M4
F2.2	Interpréteur de bytecode	Must	F2.1	M4
F2.3	Tas et allocation	Must	F2.2	M4
F2.4	Mark-and-Sweep	Must	F2.3, F1.4	M5
F2.5	Stop-and-Copy	Should	F2.3, F1.4	M6
F3.1	Mode stress	Should	F2.4	M5, M7
F3.2	Vérificateur de tas	Should	F2.4	M5, M7
F3.3	Statistiques de GC	Should	F2.4	M5
F4.1	Commande du compilateur	Must	F1.4	M3
F4.2	Commande de la VM	Must	F2.2	M4

L'ordre des jalons suit les dépendances : le bytecode (F1.4) doit être figé avant la VM (F2), et les stack maps doivent exister avant tout ramasse-miettes précis (F2.4).

6 Spécification du langage Toy

La grammaire complète est donnée dans l'annexe A du CDC. Cette section précise ce que chaque construction signifie et quelles règles le compilateur vérifie.

6.1 Éléments lexicaux

— Un **identifiant** commence par une lettre ou `_`, suivie de lettres, chiffres ou `_`.

— Les **mots réservés** sont : `struct fun var if else while return print new null true false int bool`.

— Un **entier littéral** est une suite de chiffres décimaux (le signe moins est l'opérateur unaire).

— Les espaces, tabulations et sauts de ligne séparent les éléments. Un commentaire commence par `//` et va jusqu'à la fin de la ligne.

6.2 Types et valeurs

Type	Description
`int`	entier signé sur 64 bits ; le dépassement de capacité boucle (arithmétique modulo 2^{64}) ; `/` tronque vers zéro et le signe de `\%` est celui du dividende
`bool`	`true` ou `false`, représentés par 1 et 0
struct `S`	type référence ; les champs ont des types `int`, `bool`, struct ou tableau ; une struct peut se référencer elle-même ou référencer une struct déclarée plus loin
tableau `T[]`	type référence ; `T` est `int`, `bool` ou une struct ; longueur fixée à la création ; indices de 0 à $n-1$
`null`	référence nulle, utilisable pour tout type struct ou tableau

6.3 Déclarations

— Les structs et les fonctions sont globales ; leur ordre de déclaration est indifférent.

— Une fonction a zéro ou plusieurs paramètres typés et **un type de retour obligatoire** (il n'existe pas de type `void`).

— Le programme doit contenir `fun main(): int` sans paramètre. C'est le point d'entrée.

— Une variable locale est visible de sa déclaration jusqu'à la fin du bloc qui la contient. Redéclarer un nom déjà visible est une erreur (pas de masquage).

— Sans initialiseur, une variable vaut `0`, `false` ou `null` selon son type. Il en va de même des champs d'un objet créé par `new` et des éléments d'un tableau créé par `new T[n]`.

6.4 Règles de typage

On note $S \preceq T$ la compatibilité de S avec T : $S = T$, ou S est le type de `null` et T est une struct ou un tableau.

Réf.	Construction	Règle
T1	`var x: T = e;`	le type de `e` est compatible avec `T`
T2	`lv = e;`	`lv` est une variable, un champ ou un élément de tableau ; le type de `e` est compatible avec celui de `lv`
T3	`if (c)`, `while (c)`	`c` est de type `bool`
T4	`return e;`	le type de `e` est compatible avec le type de retour ; `return;` sans valeur est interdit
T5	`f(a1, ..., an)`	`f` existe, `n` égale le nombre de paramètres, chaque `ai` est compatible avec le paramètre correspondant
T6	`+ - * / \%` et `-` unaire	opérandes `int`, résultat `int`
T7	`< <= > >=`	opérandes `int`, résultat `bool`
T8	`== !=`	deux `int`, ou deux `bool`, ou deux références dont l'une est compatible avec l'autre ; résultat `bool`
T9	`&& || !`	opérandes `bool`, résultat `bool`
T10	`a.f`	`a` est une struct qui possède le champ `f` ; le type est celui du champ
T11	`a[i]`	`a` est un tableau et `i` est `int` ; le type est celui des éléments
T12	`new S`, `new T[n]`	`S` est une struct ; `n` est `int` ; `new T[n]` a le type `T[]`
T13	`print(e);`	`e` est de type `int` ou `bool`
T14	instruction expression	l'expression est un appel de fonction
T15	noms et types	toute variable, fonction, struct et tout champ utilisé est déclaré
T16	fonction	tout chemin d'exécution se termine par `return`

6.5 Sémantique d'exécution

— Les expressions sont évaluées de gauche à droite, arguments d'appel compris.

— Les opérateurs `&&` et `||` sont paresseux : l'opérande de droite n'est évalué que si nécessaire.

— `==` et `!=` sur des références comparent l'identité (même objet), pas le contenu.

— `print(e);` écrit la valeur sur la sortie standard, suivie d'un saut de ligne : un entier en décimal, un booléen sous la forme `true` ou `false`.

— Les erreurs d'exécution sont : division ou modulo par zéro, déréférencement de `null`, indice hors bornes, taille de tableau négative, profondeur d'appel excessive, mémoire épuisée.

7 Spécification du compilateur (F1, F4.1)

7.1 Interface en ligne de commande

```
python3 compiler/toyc.py <source.toy> [-o <sortie.tbc>] [--dump]
```

— Sans `-o`, la sortie porte le nom de la source avec l'extension `.tbc`.

— `-dump` affiche le bytecode lisible (désassemblage, F1.5) sur la sortie standard.

— Code de sortie : 0 si la compilation réussit, 1 sinon.

— En cas d'erreur, aucun fichier `.tbc` n'est produit, et un message est écrit sur la sortie d'erreur.

7.2 Chaîne de traitement

ID	Passe	Entrée	Sortie
F1.1	Lexer	texte source	liste de jetons (type, valeur, ligne, colonne)
F1.2	Parser	jetons	arbre syntaxique (AST)
F1.3	Vérificateur de types	AST	AST annoté : type de chaque expression, indicateur « référence ou non »
F1.4	Générateur de code	AST annoté	fonctions, instructions, descripteurs de types, stack maps
F1.5	Désassembleur	fichier `.tbc` ou structures internes	texte lisible

7.3 Messages d'erreur

Le format est `fichier:ligne:colonne: message`, et les messages sont rédigés en anglais, comme dans le lexer déjà écrit. Le compilateur s'arrête à la première erreur lexicale ou syntaxique ; le vérificateur de types peut en signaler plusieurs.

Cas	Exemple de message
Caractère inconnu	`unexpected character '@'`
Jeton inattendu	`expected ';', found '}'`
Variable ou fonction inconnue	`undefined variable 'x'`
Champ inconnu	`unknown field 'next2' in struct 'Node'`
Types incompatibles	`type mismatch: expected int, found bool`
Mauvais nombre d'arguments	`wrong number of arguments for 'push': expected 2, found 1`
Déclaration en double	`duplicate declaration of 'x'`
Absence de `main`	`missing function 'main'`
Chemin sans `return`	`missing return in function 'f'`

8 Spécification du bytecode et du fichier .tbc (F1.4, F2.1)

8.1 Modèle d'exécution

— La VM est une machine à pile. Toutes les valeurs occupent 64 bits : entier, booléen (0 ou 1) ou référence (adresse d'un objet, 0 pour `null`).

— Chaque appel de fonction crée un **cadre** : d'abord les variables locales (paramètres en premier, dans l'ordre), puis la pile d'opérandes de la fonction.

— À l'entrée d'une fonction, les locales qui ne sont pas des paramètres sont mises à zéro.

— Chaque déclaration de variable reçoit son **propre emplacement** pour toute la fonction, qui ne change jamais de type. C'est ce qui permet de décrire chaque emplacement par un seul bit « référence ou non ».

— Une instruction occupe 8 octets : l'octet 0 est le code opération, les octets 4 à 7 sont l'opérande entier signé sur 32 bits (0 s'il n'y en a pas). Les cibles de saut sont des *indices d'instruction* dans le tableau de code.

8.2 Jeu d'instructions (proposé, arrêté à M3)

Notation de pile : $\ldots\ a\ b \to \ldots\ r$ signifie que a et b (sommet) sont dépilés et que r est empilé.

Instruction	Opérande	Pile	Comportement
`PUSH_INT`	`n`	$\ldots \to \ldots\ n$	empile l'entier `n` (aussi pour `true` = 1 et `false` = 0)
`PUSH_NULL`	—	$\ldots \to \ldots\ 0$	empile la référence nulle
`POP`	—	$\ldots\ a \to \ldots$	supprime le sommet (résultat d'un appel non utilisé)
`LOAD_LOCAL`	`i`	$\ldots \to \ldots\ v$	empile la locale numéro `i`
`STORE_LOCAL`	`i`	$\ldots\ v \to \ldots$	range le sommet dans la locale `i`
`ADD SUB MUL`	—	$\ldots\ a\ b \to \ldots\ r$	arithmétique sur 64 bits, modulo 2^{64}
`DIV MOD`	—	$\ldots\ a\ b \to \ldots\ r$	erreur d'exécution si $b = 0$
`NEG`	—	$\ldots\ a \to \ldots\ -a$	opposé
`NOT`	—	$\ldots\ a \to \ldots\ r$	$r = 1$ si $a = 0$, sinon 0
`EQ NE LT LE GT GE`	—	$\ldots\ a\ b \to \ldots\ r$	comparaison signée ; r vaut 0 ou 1
`JMP`	`t`	inchangée	saute à l'instruction `t`
`JZ`	`t`	$\ldots\ c \to \ldots$	dépile `c` ; saute à `t` si $c = 0$
`CALL`	`f`	$\ldots\ a_1 \ldots a_n \to \ldots\ r$	appelle la fonction `f` ; `n` vient de la table des fonctions ; les arguments deviennent les premières locales de l'appelé
`RET`	—	$\ldots\ v \to$	termine la fonction et empile `v` chez l'appelant ; dans `main`, arrête le programme

Instruction	Opérande	Pile	Comportement
`NEW`	`t`	$\ldots \to \ldots\ p$	alloue un objet du type `t`, champs à zéro
`NEW_ARRAY`	`t`	$\ldots\ n \to \ldots\ p$	alloue un tableau de `n` éléments à zéro ; erreur si $n < 0$
`GETFIELD`	`i`	$\ldots\ p \to \ldots\ v$	lit le champ `i` ; erreur si $p =$ `null`
`PUTFIELD`	`i`	$\ldots\ p\ v \to \ldots$	écrit `v` dans le champ `i` ; erreur si $p =$ `null`
`ALOAD`	—	$\ldots\ p\ i \to \ldots\ v$	lit l'élément `i` ; erreur si $p =$ `null` ou `i` hors bornes
`ASTORE`	—	$\ldots\ p\ i\ v \to \ldots$	écrit l'élément `i` ; mêmes erreurs
`PRINT_INT`	—	$\ldots\ a \to \ldots$	affiche l'entier suivi d'un saut de ligne
`PRINT_BOOL`	—	$\ldots\ a \to \ldots$	affiche `true` ou `false` suivi d'un saut de ligne

Les opérateurs `&&` et `||` n'ont pas d'instruction propre : le compilateur les traduit avec `JZ` et `JMP` pour obtenir l'évaluation paresseuse.

8.3 Format du fichier .tbc

Section	Contenu
En-tête	4 octets `TBC1` (identifiant et version du format)
Types	entier 32 bits `n_types`, puis par type : 1 octet de genre (1 = struct, 2 = tableau de valeurs, 3 = tableau de références), un entier 32 bits `nb_champs`, et un masque 64 bits des champs qui sont des références (le bit i vaut 1 si le champ i est une référence). Les `type_id` commencent à 1 ; 0 est invalide
Fonctions	entier 32 bits `n_fonctions`, puis par fonction : indice de la première instruction, nombre de paramètres, nombre de locales, longueur et octets du nom ; puis l'indice de `main`
Code	entier 32 bits `n_instructions`, puis les instructions de 8 octets
Stack maps	entier 32 bits `n_cartes`, puis par carte : indice d'instruction, profondeur de pile, masque 64 bits des locales qui sont des références, masque 64 bits des emplacements de pile qui sont des références

8.4 Stack maps

— Il y a une carte pour chaque **point de sûreté** : chaque instruction `NEW`, `NEW_ARRAY` et `CALL`.

— La carte décrit l'état du cadre **juste après le dépilement des opérandes** de l'instruction. Pour `NEW_ARRAY`, la taille n'est donc plus sur la pile ; pour `CALL`, les arguments ne sont plus sur la pile de l'appelant, car ils sont devenus les premières locales de l'appelé, décrites par la carte de l'appelé.

— Un cadre suspendu mémorise l'indice de son instruction `CALL` ; c'est la clé qui sert à retrouver sa carte.

— Le bit i d'un masque vaut 1 si l'emplacement i contient une référence (éventuellement `null`), 0 sinon.

9 Spécification de la VM, du tas et du GC (F2, F3, F4.2)

9.1 Interface en ligne de commande

```
vm/gcvm <programme.tbc> [--gc=ms|copy] [--heap=<octets>]
                        [--stress] [--verify] [--gc-stats]
```

Option	Effet
`-gc=ms` (défaut)	utilise Mark-and-Sweep
`-gc=copy`	utilise Stop-and-Copy (Cheney)
`-heap=<octets>`	taille totale du tas ; défaut 8 388 608 (8 Mo). Avec `copy`, le tas est partagé en deux demi-espaces égaux, pour comparer les deux GC à mémoire égale
`-stress`	collecte avant chaque allocation, et vérificateur de tas après chaque collecte
`-verify`	vérificateur de tas après chaque collecte, sans forcer une collecte à chaque allocation
`-gc-stats`	affiche les statistiques sur la sortie d'erreur à la fin

9.2 Codes de sortie

Code	Signification
0 à 99	fin normale : valeur retournée par `main` (les valeurs supérieures à 99 sont déconseillées)
1	option ou fichier invalide en ligne de commande
100	erreur d'exécution (division par zéro, `null`, indice, etc.)
101	mémoire insuffisante après une collecte
102	anomalie détectée par le vérificateur de tas
103	fichier `.tbc` invalide

9.3 Disposition des objets (proposé, arrêté à M4)

Un emplacement (*slot*) fait 8 octets. Chaque objet commence par un en-tête de 16 octets suivi de ses emplacements.

Élément	Contenu
Mot 0 de l'en-tête	bits 0 à 31 : `type_id` ; bit 32 : marqué (Mark-and-Sweep) ; bit 33 : déplacé (Stop-and-Copy)
Mot 1 de l'en-tête	nombre d'emplacements (champs ou éléments) ; si l'objet est déplacé, nouvelle adresse (pointeur de redirection)
Emplacements	un par champ (struct) ou par élément (tableau), de 8 octets chacun, y compris pour `int` et `bool`

Les emplacements qui sont des références se trouvent grâce au descripteur de type : le masque des champs pour une struct, « aucun » pour un tableau de valeurs, « tous » pour un tableau de références.

9.4 Allocation et déclenchement du GC

— Une allocation qui ne trouve pas de place déclenche une collecte, puis réessaie une seule fois. Si elle échoue encore, le programme s'arrête avec le code 101.

— Mark-and-Sweep alloue dans une liste de blocs libres (premier bloc assez grand) ; le balayage fusionne les blocs libres voisins.

— Stop-and-Copy alloue en avançant un pointeur dans le demi-espace courant.

— Avec `-stress`, une collecte précède chaque allocation.

9.5 Racines et précision

Les racines sont uniquement les emplacements marqués « référence » dans les stack maps de tous les cadres actifs, parcourus du plus récent au plus ancien. Aucune valeur n'est interprétée comme un pointeur parce qu'elle *ressemble* à une adresse.

9.6 Mark-and-Sweep (F2.4)

1. **Marquage :** à partir des racines, marquer chaque objet atteignable en suivant ses champs référence. Le parcours utilise une pile explicite et **pas de récursion** : une liste de 1 000 000 d'éléments ne doit pas faire déborder la pile du langage C.
2. **Balayage :** parcourir le tas ; rendre libre tout objet non marqué, effacer les marques des autres, fusionner les blocs libres voisins.

9.7 Stop-and-Copy, algorithme de Cheney (F2.5)

1. Copier dans le demi-espace d'arrivée les objets directement référencés par les racines, en laissant un pointeur de redirection dans l'ancien objet, et mettre à jour chaque racine.
2. Parcourir les objets copiés avec un pointeur de balayage : copier ou rediriger chaque objet qu'ils référencent et mettre à jour le champ.
3. Quand le pointeur de balayage rejoint le pointeur d'allocation, échanger les deux demi-espaces.

9.8 Vérificateur de tas (F3.2)

Après une collecte, le vérificateur contrôle que :

— chaque racine est `null` ou désigne le début d'un objet vivant valide ;

— chaque champ référence d'un objet atteignable est `null` ou désigne un objet vivant valide ;

— chaque `type_id` d'en-tête existe dans la table des types ;

— avec Mark-and-Sweep, aucun objet n'est resté marqué et la liste libre est cohérente ; avec Stop-and-Copy, aucun pointeur ne désigne l'ancien demi-espace.

En cas d'anomalie, il affiche l'adresse et la nature de l'erreur, puis le programme s'arrête avec le code 102.

9.9 Statistiques (F3.3)

Avec `-gc-stats`, la VM écrit sur la sortie d'erreur une ligne de la forme :

```
[gc] algo=<ms|copy> collectes=<n> objets_alloues=<n> octets_alloues=<n>
     octets_recuperes=<n> temps_gc_ms=<x>
```

10 Contraintes et règles de gestion

Réf.	Règle
RG-01	Un programme contient exactement une fonction `main(): int` sans paramètre.
RG-02	Les noms de structs et de fonctions sont uniques et différents de `int` et `bool`.
RG-03	Il n'y a pas de masquage de variable : une déclaration reçoit un emplacement local propre.
RG-04	Les valeurs par défaut sont `0`, `false` et `null`.
RG-05	Il n'y a aucune conversion implicite entre `int` et `bool`.
RG-06	Limites de l'implémentation : une struct a au plus 64 champs, une fonction au plus 64 locales, la pile d'une fonction contient au plus 64 emplacements à chaque point de sûreté ; la profondeur d'appel est limitée à 10 000 cadres. Le compilateur signale un dépassement.
RG-07	Aucun entier n'est jamais traité comme un pointeur : seul le couple stack maps et descripteurs de types identifie les références.
RG-08	Le marquage de Mark-and-Sweep n'est pas récursif.
RG-09	Après une collecte Stop-and-Copy, toutes les racines et tous les champs référence sont mis à jour.
RG-10	Un programme valide ne provoque jamais de plantage du système : toute erreur d'exécution produit un message et un code de sortie défini.
RG-11	Pour un même programme, les deux GC produisent la même sortie standard.
RG-12	Une erreur de compilation ne laisse aucun fichier `.tbc`.

11 Traçabilité avec le cahier des charges

CDC	Fonctionnalités	Jalons	Vérification
US1	F1.1 à F1.4	M1 à M3	`list.toy` et `arrays.toy` compilent
US2	F1.1 à F1.3	M1, M2	tests négatifs : un programme invalide par erreur de la section 7
US3	F2.2	M4	sorties attendues des programmes de test
US4	F2.3, F2.4	M4, M5	`list.toy` dans un tas borné
US5	F4.2	M4 à M6	options `-gc`, `-heap`, `-gc-stats`
US6	F3.1, F3.2	M5, M7	suite de tests complète avec `-stress`
US7	F2.2	M4	un test par erreur d'exécution

A Illustration : une fonction compilée

Pour la fonction `push` de `tests/list.toy` :

```
struct Node { val: int; next: Node; }

fun push(head: Node, v: int): Node {
    var n: Node = new Node;
    n.val = v;
    n.next = head;
    return n;
}
```

Les locales sont : 0 = `head`, 1 = `v`, 2 = `n`. Les champs de `Node` sont : 0 = `val` et 1 = `next`, donc son masque de références vaut 10_2 (seul `next` est une référence). Le bytecode produit est :

```
 0  NEW          Node       ; carte : locales 101, pile vide
 1  STORE_LOCAL  2          ; n = new Node
 2  LOAD_LOCAL   2
 3  LOAD_LOCAL   1
 4  PUTFIELD     0          ; n.val = v
 5  LOAD_LOCAL   2
 6  LOAD_LOCAL   0
 7  PUTFIELD     1          ; n.next = head
 8  LOAD_LOCAL   2
 9  RET                     ; retourne n
```

La stack map de l'instruction 0 (`NEW`) indique : profondeur de pile 0 ; masque des locales 101_2 (bit 0 : `head`, bit 2 : `n` sont des références, bit 1 : `v` est un entier). Si une collecte a lieu pendant ce `NEW`, le GC traite `head` et `n` (encore `null`) comme racines et ignore `v`.